

Change Request (CR-02)

Certificate-to-Training Expertise Linking + Visual Skill Tree

Base SRS: Sulfri Trainer Portal — Personal Brand Edition v1.0 (and CR-01 where applicable)

CR ID: CR-02

Type: Functional Enhancement (Brand Credibility + Visual UX)

Priority: High

Target Version: v1.2 (or v1.1 + Addendum)

1. Executive Summary

This CR introduces two connected enhancements:

1) **Certificate/Badge-to-Training Expertise Hook**

Badges/certificates imported via Credly (embed/manual or JSON auto-sync) must be linked to **Training Expertise** areas (e.g., Data Analytics, Cybersecurity, AI Productivity, Project Management).

2) **Training Expertise Visualization as a Skill Tree**

The Training Expertise section will be visualized as an interactive **skill tree** (nodes + branches), showing progression from core domains to sub-topics, with evidence signals from badges/certificates.

This strengthens credibility by making it obvious how credentials support the services/training areas offered.

2. Scope of Change

2.1 In Scope

- A structured **Training Expertise** module (if not already present) with:
 - Expertise categories
 - Topics/sub-topics
 - Proficiency level (admin-controlled)
 - Evidence links to badges/certificates
- New public page/section:
 - /expertise (Skill Tree view)

- Admin management pages:
 - /admin/expertise (tree editor)
 - /admin/mappings (badge/cert ↔ expertise mapping)
- Rules to associate Credly badges to expertise:
 - Manual mapping
 - Optional rules-based mapping (keywords)

2.2 Out of Scope (Later)

- Auto-generating the entire tree via AI
 - Multi-user (student) skill trees
 - Paid credential verification beyond Credly
-

3. Functional Requirements

3.1 Training Expertise Data Model

FR-46

System must support a **Training Expertise** structure with hierarchical levels: - Domain (Level 1) → Track (Level 2) → Topic (Level 3) → Subtopic (Level 4 optional)

FR-47

Admin must be able to set a node's: - Title - Description - Category (Domain) - Level / Depth - Display order - Visibility (Public/Hidden) - Proficiency Level (e.g., Foundation / Intermediate / Advanced / Specialist)

FR-48

Admin must be able to attach deliverables to a node (optional): - Course outlines - Sample materials - Portfolio case studies

3.2 Hook Certificates/Badges to Training Expertise

FR-49

Admin must be able to link a badge/certificate to one or more expertise nodes.

FR-50

On badge detail pages (/badges/{slug}), the UI must display: - “Relevant Training Expertise” list (linked nodes)

FR-51

On expertise node detail (via skill tree), the UI must display: - Supporting badges/certificates (embedded view where applicable) - Verification links

FR-52

If Auto-Sync JSON Mode (CR-01) is enabled, system must allow: - Post-sync mapping workflow (new badges appear as “Unmapped”)

3.3 Skill Tree Visualization (Public)

FR-53

The Training Expertise section must be visualized as a **Skill Tree**: - Nodes represent domains/topics - Lines show dependency/progression - Expand/collapse per domain

FR-54

Skill Tree must support: - Zoom/pan (desktop) - Swipe/drag (mobile) - Search (jump to node) - Filter by domain

FR-55

Each node must show a credibility indicator derived from linked badges: - Example: “Verified Evidence: 3 badges”

FR-56

Clicking a node must open a detail panel showing: - Node description - Related courses offered - Supporting badges/certificates (with verify links) - Optional: downloadable outline/sample

FR-57

Skill Tree must degrade gracefully on low-power devices: - Switch to “List View” toggle (tree ↔ list)

3.4 Admin Skill Tree Management

FR-58

Admin must be able to manage the tree through a structured editor: - Add node - Edit node - Move node (change parent) - Reorder nodes - Hide/unhide

FR-59

Admin must be able to define **edges** (relationships) between nodes: - Parent-child (default)
- Optional cross-links (advanced)

FR-60

System must validate tree integrity: - No circular dependencies - Maximum depth configurable (default 4)

3.5 Rules-Based Mapping (Optional, Recommended)

FR-61

Admin can define simple mapping rules to auto-suggest expertise links for new badges: - Keyword contains (title/issuer) - Category match

FR-62

System must present suggestions for admin approval (never auto-publish mappings without admin confirmation).

4. Database Amendments

4.1 expertise_nodes

- id (PK)
- title
- slug (unique)
- description (nullable)
- domain (string or FK)
- parent_id (nullable FK to expertise_nodes.id)
- depth (int)
- proficiency_level (enum)
- visibility (PUBLIC | HIDDEN)
- display_order (int)
- created_at
- updated_at

4.2 expertise_edges (optional if using only parent_id)

- id (PK)
- from_node_id (FK)
- to_node_id (FK)
- edge_type (PARENT_CHILD | CROSS_LINK)

4.3 badge_expertise_map

- badge_id (FK to badges)
- expertise_node_id (FK)
- mapping_source (MANUAL | SUGGESTED)
- created_at

4.4 expertise_assets (optional)

- id (PK)
 - expertise_node_id (FK)
 - asset_type (OUTLINE | SAMPLE | CASE_STUDY | LINK)
 - title
 - url
-

5. API Amendments

Public

- GET /api/expertise/tree (returns nodes + edges)
- GET /api/expertise/{slug}
- GET /api/expertise/{slug}/badges

Admin (Protected)

- CRUD POST/PATCH/DELETE /api/admin/expertise-nodes
 - POST /api/admin/expertise/move-node
 - CRUD POST/PATCH/DELETE /api/admin/badge-expertise-map
 - POST /api/admin/expertise/rules (optional)
-

6. UX Requirements

6.1 Public Expertise Page

- Executive, clean style consistent with MSH brand
- Default view: Skill Tree
- Toggle: Tree View / List View
- Search bar
- Domain filter chips

6.2 Node Detail Panel

- Slide-over panel (desktop) / bottom sheet (mobile)
 - Shows:
 - Node title + proficiency
 - Description
 - Supporting badges (embedded where possible)
 - Verify links
 - Related courses/assets
-

7. Non-Functional Requirements

7.1 Performance

- Tree data must be cached server-side
- Render virtualization for large trees
- Lazy-load badge embeds inside node panel only (not for every node)

7.2 Security

- Same embed safety rules from CR-01 apply
- Admin-only endpoints protected
- Validate URLs for assets

7.3 Accessibility

- Keyboard navigation for list view
 - Focus management for panels
-

8. Acceptance Criteria

1. Admin can create expertise nodes and build a tree.
 2. Admin can link a Credly badge to one or more expertise nodes.
 3. Public skill tree displays nodes and indicates evidence count.
 4. Clicking a node shows supporting badges + verification links.
 5. New badges from Auto-Sync appear as “Unmapped” until admin links them.
 6. List view works as fallback and remains usable on mobile.
-

9. Implementation Notes (Recommended Approach)

- Store tree as adjacency list (parent_id) for simplicity.
 - Generate tree JSON via API for front-end rendering.
 - Use a dedicated front-end component for rendering (supports zoom/pan).
 - Only embed badges when needed (detail panel).
-

10. Versioning Recommendation

- Keep CR-02 as addendum for review.
- After approval, merge into master SRS as:
 - New module under Functional Requirements
 - New database tables
 - New routes and APIs